

NPTEL Week 11

ASSIGNMENT 11 SOLUTIONS & EXPLANATIONS

PROGRAMMING SOLUTIONS

1. Review Word Count

// PYTHON

```
search, *reviews = input().split()
print(reviews.count(search), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
  el.focus();
  document.execCommand('selectAll', false, null);
  document.execCommand('insertText', false, `search, *reviews = input().split()
print(reviews.count(search), end='')`);
}
```

2. Hotel Stay Duration

// PYTHON

```
from datetime import datetime
d1, d2 = [datetime.strptime(x, '%Y-%m-%d') for x in input().split()]
print(abs((d2 - d1).days), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
  el.focus();
  document.execCommand('selectAll', false, null);
  document.execCommand('insertText', false, `from datetime import datetime
d1, d2 = [datetime.strptime(x, '%Y-%m-%d') for x in input().split()]
print(abs((d2 - d1).days), end='')`);
}
```

3. Sundays in a Month

// PYTHON

```
import calendar
year, month = [int(x) for x in input().split()]
# week[6] is Sunday; 0 means that day belongs to another month
sundays = sum(1 for week in calendar.monthcalendar(year, month) if week[6] != 0)
print(sundays, end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `import calendar
year, month = [int(x) for x in input().split()]
# week[6] is Sunday; 0 means that day belongs to another month
sundays = sum(1 for week in calendar.monthcalendar(year, month) if week[6] != 0)
print(sundays, end='')`);
}
```

MCQ & MSQ SOLUTIONS WITH EXPLANATIONS

Scenario 1: Semester Planner

Q01 **calendar.month()**

Explanation: Line 9 prints `calendar.month(year, month)`, which returns the full month calendar as a formatted multi-line string.

Q02 **The index representing the weekday**

Explanation: `calendar.weekday(year, month, day)` returns an integer from 0 to 6, where 0 is Monday and 6 is Sunday. It does not return the name.

Q03 **To convert the weekday index into its corresponding weekday name**

Explanation: The list is ordered Monday to Sunday, matching the indices returned by `calendar.weekday()`, so `days[weekday]` gives the name.

Q04 **print("Event Day:", days[weekday])**

Explanation: Line 15 uses the index from Line 11 to look up and print the weekday name. The other options print the calendar or only compute the index.

Q05 **Selected Options:** MSQ

—The user enters the year, month, and day.

—The calendar is displayed before the weekday is printed.

Explanation: Lines 3–5 read the year, month and day, and the calendar (Line 9) is printed before the event day (Line 15). `calendar.weekday()` returns an index, not a name, and `days` gives the weekday name, not the month name.

Scenario 2: Meeting Schedule Planner

Q06 To check whether the given year is a leap year

Explanation: `calendar.isleap(year)` returns `True` for leap years and `False` otherwise, which decides between the two print statements on Lines 10 and 12.

Q07 A list containing the weeks of the month

Explanation: `calendar.monthcalendar(year, month)` returns a list of weeks, each a list of 7 day numbers (Monday to Sunday), with 0 for days outside the month.

Q08 To count only the Mondays that exist in the selected month

Explanation: `week[0]` is the Monday slot. In the first week it may be 0 because that Monday belongs to the previous month, so it must not be counted.

Q09 monday_count

Explanation: It starts at 0 on Line 15, is incremented on Line 19 for every valid Monday, and is printed on Line 21.

Q10 Selected Options: MSQ

—`calendar.monthcalendar()` returns a list of weeks.

—The calendar is displayed before the number of Mondays is printed.

Explanation: `monthcalendar()` returns a list of weeks, and the calendar (Line 7) is printed before the count (Line 21). The program counts Mondays, not days, and `isleap()` returns a Boolean, not the number of days in February.

Scenario 3: Shift Allocation Calendar

Q11 An integer representing the weekday of the first day of the month

Explanation: `calendar.weekday(year, month, 1)` returns 0–6 for the weekday of the 1st. Line 23 later converts it to a name using `day_names[first_day]`.

Q12 To count Saturdays and Sundays that belong to the selected month

Explanation: `week[5]` is Saturday and `week[6]` is Sunday. A value of 0 is padding for a day from a neighbouring month, so only non-zero entries are counted.

Q13 It stores a two-dimensional list where each inner list represents one week.

Explanation: For example, October 2026 gives `[[0,0,0,1,2,3,4], [5,...,11], ..., [26,27,28,29,30,31,0]]`: one inner list of 7 days per week.

Q14 9

Explanation: Each valid Saturday and each valid Sunday adds 1 to `weekend_count`, so $5 + 4 = 9$.

Q15 Selected Options: MSQ

—`calendar.weekday(year, month, 1)` is used to determine the weekday of the first day of the month.

—The program displays the calendar before counting the weekends.

Explanation: Line 10 calls `calendar.weekday(year, month, 1)`, and the calendar (Line 7) is printed before the loop counts weekends. Each element of `weeks` is a list, not a string, and `weekend_count` can reach 10 (a 31-day month starting on Saturday has 5 Saturdays and 5 Sundays).

Scenario 4: Training Session Scheduler

Q16 The dates that fall on Fridays in the selected month

Explanation: Line 14 appends `week[4]`, which is the actual date number in the Friday column, e.g. `[3, 10, 17, 24, 31]`.

Q17 To ensure the Friday belongs to the selected month

Explanation: If the month starts after Friday, the first week's Friday slot is 0 (it belongs to the previous month). The check stops a fake 0 from being added to `fridays`.

Q18 It returns the last element in the `fridays` list.

Explanation: Negative indexing counts from the end, so `fridays[-1]` is the last Friday of the month. It does not remove anything.

Q19 31

Explanation: With `fridays = [3, 10, 17, 24, 31]`, the length is 5 (≥ 4), so Line 19 prints `fridays[-1]`, which is 31.

Q20 Selected Options: MSQ

—The list `fridays` stores the dates that fall on Fridays.

—The program displays the last Friday only if at least four Fridays are present in the month.

Explanation: `fridays` stores the Friday dates, and Line 18 only prints the last Friday when `len(fridays) >= 4`. `monthcalendar()` returns a list, not a string, and `week[4]` is the Friday column, not the fifth week.
